

Databases Resit: Full Topic Guide (15 questions)

For each question type: **Know** (the concepts), **Do** (the step-by-step method), and a **Worked example** taken from your own exam paper with the answer.

Where to spend tonight (by recoverable points): BCNF synthesis (9), two-phase locking (8), ER diagram (8), and the four SQL query tasks (16 combined). You already had conditions, conflict-serializability, and most of canonical FDs.

1. Entity Relationship Diagrams

Know. Entities (rectangles), attributes (ovals, key underlined), relationships (diamonds). Cardinalities use UML look-across (min..max): the label nearest an entity says how many of *that* entity relate to **one** instance of the other side. Weak entity = double rectangle, partial key dashed-underlined, identified through a double-diamond identifying relationship to its owner. Recursive relationships need role labels. Avoid artificial keys unless forced.

Do.

1. Read each sentence and extract: an entity, an attribute, a relationship, or a cardinality.
2. For each “unique within X but not globally” attribute, make the entity weak, owned by X.
3. Watch the trap: if a relationship needs its *own* relationship (e.g. a work assignment that is supervised), promote it to a weak/associative entity, then attach the extra relationship to it.
4. Put every cardinality on the correct side (look-across). This is the single most common place to lose marks.
5. Note any constraint ER cannot express (e.g. “supervisor must be a different person”) as a written comment.

Worked example (the company/employee scenario).

- Branch (name key, address); Department is **weak**, owned by Branch, partial key name ; Employee (id key, firstName, lastName, dateOfBirth, optional middleName); WorkAssignment is **weak**, owned by Employee + Department, attribute hoursPerWeek.
- consistsOf (Branch 1..1 / Department 1..); isHeadedBy (Employee 1..1 / Department 0..); hasBoss recursive (boss 0..1 / subordinate 0..); doneBy (Employee 1..1 / WorkAssignment

0.); atDepartment (Department 1..1 / WorkAssignment 0.); supervisedBy (Employee 0..1 / WorkAssignment 0.).

- WorkAssignment's identity (`Employee.id`, `Department key`) is what enforces "at most one assignment per department per employee" with no artificial key.
-

2. Relational Model (translate an ER diagram to a relational schema)

Know. Translation rules:

- Strong entity: one relation, attributes as columns, primary key = entity key.
- Weak entity: include the owner's key in a composite primary key, plus a foreign key to the owner.
- 1:N relationship: put the key of the "1" side as a foreign key in the "N" side's relation (and any relationship attributes go there too). No separate relation.
- 1:1 relationship: foreign key in either relation (put it on the mandatory side, optionally `unique`).
- N:M relationship: a separate relation whose primary key is both sides' keys combined (each also a foreign key), plus relationship attributes.
- isA / specialization: either one relation per subclass carrying the superclass key, or superclass relation plus subclass relations with foreign keys.
- Comment on: candidate keys (`unique`), `not null` for mandatory participation and required foreign keys, and which **minimum** cardinalities a plain schema cannot enforce.

Do.

1. One relation per strong entity, then weak entities, then relationships by their cardinality class above.
2. Decide every foreign key's nullability from the min cardinality (mandatory participation, exactly-one, becomes `not null`).
3. List the constraints a relational DB can check (keys, `unique` , `not null` , referential integrity) and explicitly name the ones it cannot (a minimum of one, totality/disjointness of a hierarchy).

Worked example (Person / Customer / Employee / Order from the exam). Cardinalities:

ordered (Customer 1..1 / Order 1..) means one customer to many orders, each order exactly one customer; handled (Order 0.. / Employee 0..1) means one employee to many orders, each order at most one employee; `handled` has attribute `time`.

<code>Person(id, name, phone)</code>	<code>key: id</code>
<code>Customer(id)</code>	<code>key: id; id -> Person</code>
<code>Employee(id, salary)</code>	<code>key: id; id -> Employee... (id -</code>
<code>Order(orderNr, customer_id, employee_id, time)</code>	<code>key: orderNr</code>
	<code>customer_id -> Customer (not nul</code>
	<code>employee_id -> Employee (nullabl</code>

Comment: `id` and `orderNr` are unique (primary keys). `customer_id not null` enforces the "each order has exactly one customer." `employee_id nullable` enforces the 0..1. Cannot enforce in a plain schema: that each customer has at least one order (the minimum of 1), and the totality/disjointness of the Person specialization.

3. Canonical (minimal) functional dependencies

Know. A canonical/minimal cover is an FD set that is equivalent to the original, has a single attribute on each right-hand side, no extraneous left-hand attributes, and no redundant FDs. The closure X^+ is everything derivable from X .

Do (in this order).

1. **Singleton right sides:** split every $X \rightarrow A, B$ into $X \rightarrow A, X \rightarrow B$.
2. **Left-reduce:** for each $X \rightarrow A$ with $|X| > 1$ and each attribute B in X , if A is in $(X \text{ minus } B)^+$ under the current set, drop B .
3. **Remove redundant FDs:** for each $X \rightarrow A$, compute X^+ using all the *other* FDs; if A is in it, delete that FD.
4. Minimal covers are not unique, but yours must be equivalent to the original (check by confirming a key still closes to everything).

Worked example (attributes A,B,C,D,E). Original: $AB \rightarrow CE, D \rightarrow E, BE \rightarrow CD, B \rightarrow ACE, ADE \rightarrow C$.

- B closes to everything, so B is a key. After splitting and left-reducing, $AB \rightarrow C/E$ and $BE \rightarrow C$ collapse into $B \rightarrow C/E$, $BE \rightarrow D$ becomes $B \rightarrow D$, and $ADE \rightarrow C$ reduces to $AD \rightarrow C$.
- Remove redundancy: $B \rightarrow C$ is implied by $B \rightarrow A, B \rightarrow D, AD \rightarrow C$; and $B \rightarrow E$ is implied by $B \rightarrow D, D \rightarrow E$. Drop both.

Minimal cover:

$B \rightarrow A$
 $B \rightarrow D$
 $D \rightarrow E$
 $A, D \rightarrow C$

(Your earlier attempt was missing $AD \rightarrow C$ and kept an extra $B \rightarrow C$, which is why it scored 6/7. The set must let you derive C from A and D.)

4. Minimal keys (find all candidate keys)

Know. A candidate (minimal) key X has $X^+ = \text{all attributes}$ and no proper subset does.

Pre-classification: an attribute that appears **only on left-hand sides** (or in no FD) must be in every key; an attribute that appears **only on right-hand sides** is never in any minimal key; attributes that never appear on any left-hand side cannot be in a key (they determine nothing).

Do.

1. Classify attributes (must-be-in, cannot-be-in, optional).
2. Compute the closure of the must-be-in set. If it is already everything, that is the unique key.
3. Otherwise, add optional attributes in increasing-size combinations; test each closure for "everything"; once a set is a key, skip all its supersets (not minimal).

Worked example (attributes A,B,C,D). FDs: $BC \rightarrow A$, $ABD \rightarrow C$, $BC \rightarrow D$, $C \rightarrow AB$, $AD \rightarrow B$.

- Every attribute appears on both sides, so no forced in/out from the simple rule.
- $C^+ : C$ gives A,B; then BC gives D, so $C^+ = ABCD$. So **{C} is a key.**
- Any minimal key without C must come from {A,B,D}. $(AD)^+ : AD$ gives B, then ABD gives C, so $(AD)^+ = ABCD$. So **{A,D} is a key.** Singletons A, B, D and pairs AB, BD do not close to everything; ABD contains AD so is not minimal.

Answer:

$\{C\}$
 $\{A, D\}$

5. Boyce-Codd Normal Form (synthesis algorithm)

The question (as it appears). Attributes {A, B, C, D, E, F, G}. Original FDs: $C \rightarrow B, G$; $A \rightarrow B, E$; $A, F \rightarrow G$; $A, F \rightarrow C, E$; $B, D, F \rightarrow C$; $C, D \rightarrow A, E, G$. The exam has already done the first step for you and gives the canonical (minimal) cover:

$C, D \rightarrow A$
 $C \rightarrow B$
 $C \rightarrow G$
 $A \rightarrow B$
 $B, D, F \rightarrow C$
 $A, F \rightarrow C$
 $A \rightarrow E$

Task: apply the synthesis algorithm to derive a Boyce-Codd normal form for $R(A, B, C, D, E, F, G)$. You answer in **two fields**: (1) the functional dependencies used for splitting (including those whose left-hand side is a key), and (2) the splitting steps, one per line, in the form `splitting R(A,B,C,D) into (A,B,C) and (A,D)` (relation names can be omitted).

Know. Even though it is called "synthesis," the required answer is a sequence of binary **splits** (top-down decomposition). To reach BCNF you repeatedly split any relation that has a non-trivial FD $X \rightarrow Y$ whose left-hand side X is **not a superkey** of that relation. A split on $X \rightarrow Y$ produces X^+ (the closure, one relation) and X together with everything not in X^+ (the other). The split is lossless because the shared attributes (X) form a key of the X^+ side. BCNF decomposition is always lossless but may drop some dependencies, which is allowed.

Do.

1. Compute the closure of each candidate left-hand side inside the current relation.
2. Pick an FD that violates BCNF (its LHS is not a superkey) and split on it.
3. Recurse on both halves until every relation is in BCNF (every non-trivial FD's LHS is a superkey, or the relation has no non-trivial FD).
4. Collect the FDs you split on; in each resulting relation that FD's LHS is a key.

Worked answer. Keys of R are CDF , ADF , BDF (D and F sit in every key). Every cover FD except $B, D, F \rightarrow C$ (whose LHS is a key of R) has a left-hand side that is not a superkey, so those are the ones available to split on. One correct decomposition:

Functional dependencies used for splitting (LHS becomes a key of its relation):

C → B
C → G
A → E
C, D → A

Splitting steps:

splitting R(A,B,C,D,E,F,G) into (B,C,G) and (A,C,D,E,F)
splitting (A,C,D,E,F) into (A,E) and (A,C,D,F)
splitting (A,C,D,F) into (A,C,D) and (C,D,F)

Resulting relations, all in BCNF: (B,C,G) key C, (A,E) key A, (A,C,D) key CD, (C,D,F) key CDF. Lossless, because each split shares a key of one side. It does not preserve every dependency (A→B, A,F→C, B,D,F→C end up split across relations); for BCNF that is acceptable and expected. BCNF decompositions are not unique, so any split order that is lossless and leaves every part in BCNF also earns full marks.

6. Anomalies (bonus)

Know. All three come from the same cause: a non-key dependency $x \rightarrow y$ forces Y-values to be stored redundantly (repeated in every row sharing the same X). Normal forms remove this. Anchor with a relation like Employee(empId, deptId, deptName, deptLocation) where deptId → deptName, deptLocation but deptId is not a key.

Do. Name each, give the mechanism, give a one-line example, and tie back to redundancy.

- **Insertion anomaly:** you cannot record a department's name/location until some employee is assigned to it, because empId is part of the key and cannot be null. A standalone fact has nowhere to live.
 - **Deletion anomaly:** deleting the last employee of a department also erases that department's name and location, losing information you did not intend to delete.
 - **Update anomaly:** because the department's location is duplicated across rows, changing it means updating every matching row; miss one and the database holds contradictory values (inconsistency).
-

7. SQL Conditions

Know. SELECT ... FROM ... WHERE ... , comparison operators, string literals in quotes,

and operator precedence: **AND binds tighter than OR**, so parenthesize OR groups.

Do. Translate the English condition to boolean algebra, parenthesize the OR, and project exactly the requested columns.

Worked example. "id and age of all persons that are male, and either at most 30 or at least 50."

```
SELECT id, age
FROM Persons
WHERE gender = 'male' AND (age <= 30 OR age >= 50);
```

8. SQL Joins

Know. Self-joins (the same table aliased multiple times), joining through a relationship/bridge table, aliasing, `DISTINCT` to remove duplicates, and that a "knows/likes" table is directional (personA is the subject).

Do.

1. Count how many copies of each table the question needs (one per role).
2. Alias them, join through the bridge table on the right id columns.
3. Add the comparison predicates; add `DISTINCT` if the same id can appear via multiple matches.

Worked example. "id of all persons who know someone younger and someone older than themselves." Persons, Knows(personA_id, personB_id), personA knows personB.

```
SELECT p.id
FROM Persons p
WHERE EXISTS (SELECT 1 FROM Knows k JOIN Persons q ON k.personB_id = q.id
              WHERE k.personA_id = p.id AND q.age < p.age)
AND EXISTS (SELECT 1 FROM Knows k JOIN Persons q ON k.personB_id = q.id
              WHERE k.personA_id = p.id AND q.age > p.age);
```

9. SQL Aggregations

Know. `GROUP BY`, aggregate functions (`AVG`, `COUNT`, `SUM`, `MIN`, `MAX`), and the key distinction: `WHERE` filters rows *before* grouping, `HAVING` filters groups *after* aggregation.

Aggregates may appear in `SELECT` and `HAVING`, never in `WHERE`.

Do. Identify the entity you aggregate *per* (that becomes the `GROUP BY` key), join to fetch the value being aggregated, then put the aggregate condition in `HAVING`.

Worked example. "id of all persons for whom the average age of everybody they like is at least 30 and at most 60." Likes(personA_id, personB_id), personA likes personB.

```
SELECT l.personA_id AS id
FROM Likes l
JOIN Persons p ON l.personB_id = p.id
GROUP BY l.personA_id
HAVING AVG(p.age) >= 30 AND AVG(p.age) <= 60;
```

10. SQL Non-Monotonic (negation / set difference)

Know. Negation patterns: `NOT EXISTS`, `NOT IN`, and `EXCEPT`. Prefer `NOT EXISTS` because `NOT IN` behaves wrongly if the subquery returns a `NULL`.

Do. Find the set that *does* satisfy the positive condition, then subtract it from the universe.

Worked example. "id of all persons that do NOT take baking classes." Classes(id, kind), TakesClasses(person_id, class_id); several classes can have kind 'baking'.

```
SELECT p.id
FROM Persons p
WHERE NOT EXISTS (
    SELECT 1 FROM TakesClasses t JOIN Classes c ON t.class_id = c.id
    WHERE t.person_id = p.id AND c.kind = 'baking');
```

11. SQL Advanced (universal quantification / division)

Know. "X relates to ALL Y" is the relational-division pattern. Rewrite it as "there is NO Y that X does NOT relate to," which becomes a **double** `NOT EXISTS`. Edge case: if there are zero Y, the condition is vacuously true (X is included).

Do. Outer `NOT EXISTS` over the "all" set; inner `NOT EXISTS` checking the relationship is missing. Decide whether to exclude the person themselves.

Worked example. "id of all persons that know everybody in Paris." Knows(personA_id,

personB_id).

```
SELECT p.id
FROM Persons p
WHERE NOT EXISTS (
  SELECT 1 FROM Persons q
  WHERE q.address = 'Paris'
  AND NOT EXISTS (
    SELECT 1 FROM Knows k
    WHERE k.personA_id = p.id AND k.personB_id = q.id));
```

Reads as: there is no Parisian q that p fails to know. (Add `AND q.id <> p.id` inside if the intended meaning excludes knowing yourself.)

12. Conflict-Serializability

Know. Two operations **conflict** if they are from different transactions, on the same item, and at least one is a write (write-write, read-write, write-read). The **precedence graph** has a node per transaction and an edge $T_i \rightarrow T_j$ whenever an operation of T_i conflicts with and comes *before* an operation of T_j on the same item. A schedule is conflict-serializable **iff its precedence graph is acyclic**; if acyclic, any topological order is an equivalent serial schedule.

Do.

1. List every conflicting pair in schedule order.
2. Draw an edge from the earlier transaction to the later one for each pair.
3. Check for a cycle. Acyclic means serializable plus a serial order; a cycle means not serializable.

Worked example (the four-transaction schedule). Conflicts produce edges $T_3 \rightarrow T_4$ (on D), $T_2 \rightarrow T_1$, $T_1 \rightarrow T_4$, $T_1 \rightarrow T_2$ (on B and C). Because $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ both appear, there is a cycle $T_1 \leftrightarrow T_2$, so the schedule is **not** conflict-serializable (leave the box unchecked).

13. Two-Phase Locking

Know. 2PL: each transaction has a growing phase (acquires locks) then a shrinking phase (releases locks); once it releases any lock it may never acquire another. The moment it holds

its last lock is its **lock point**. Lock compatibility: a shared (read) lock allows other shared locks but not an exclusive lock; an exclusive (write) lock allows nothing else on that item at the same time. 2PL produces conflict-serializable schedules.

Do (to insert lock/unlock actions for a given schedule).

1. For each item, take the strongest lock you will need before the first use (if a transaction reads then writes the same item, take the exclusive lock up front, or upgrade before the write).
2. Hold every lock until the transaction's lock point (the point after which it acquires no further lock), then release.
3. Check no two conflicting locks overlap in time on the same item; conflicting operations are already ordered by the schedule, so the locks hand off in that order. If an overlap is forced, the schedule is not 2PL-realizable (when the task says it *is*, a valid placement exists).

Worked micro-example. Schedule: $T1:R(A)$, $T1:W(A)$, $T2:R(A)$.

$T1: X\text{-lock}(A)$, $R(A)$, $W(A)$, $unlock(A)$ (lock point right after acquiring $X(A)$
 $T2: S\text{-lock}(A)$, $R(A)$, $unlock(A)$

$T1$ takes the exclusive lock before reading (because it will write), holds to its lock point, releases; $T2$ then locks and reads. The X -lock serializes $T1$ before $T2$.

14. Multi-Granularity Locking

Know. Four lock types: S, X, IS (intention shared), IX (intention exclusive). Intention locks on a coarse node signal an intent to take a finer lock below it. Memorize the compatibility matrix. The **only compatible pairs** are: IS with {IS, IX, S}; IX with {IS, IX}; S with {IS, S}; X with nothing. Everything else conflicts.

holds \ requests	IS	IX	S	X
IS	ok	ok	ok	conflict
IX	ok	ok	conflict	conflict
S	ok	conflict	ok	conflict
X	conflict	conflict	conflict	conflict

Do. For each “A conflicts with B” statement, look it up: it is true unless the pair is one of the compatible pairs above.

Worked example (the exam’s checklist). True (select): X-X, S-X, IS-X, IX-S, IX-X. False (leave unchecked): S-S, IS-S, IS-IS, IX-IS, IX-IX.

15. Database Application Programming (SQL Injection)

Know. SQL injection is a vulnerability where attacker-supplied input is spliced into a query and gets interpreted as **SQL code instead of data**, letting the attacker read, change, or destroy data or bypass authentication. Related topics in this section: embedded SQL, host variables, cursors, and prepared statements.

Do (answer all three parts).

- **What it is and how it occurs:** building queries by string-concatenating unsanitized input erases the code/data boundary. Example: `"SELECT * FROM Users WHERE name = '" + input + "'" ;` entering `' OR '1'='1` makes the `WHERE` always true (returns everyone, defeats a login). Inputs like `'; DROP TABLE Users; --` can delete data.
- **How to prevent it:** the primary defense is **prepared statements / parameterized queries**: the SQL is parsed with placeholders, then values are bound separately, so a bound value is always treated as data and can never change the command structure. State *why* this works (separation of code and data); that is the part a bare “use prepared statements” misses. Supporting measures: input validation/whitelisting, least privilege on the database account, and escaping only when parameterization is impossible.

```
SELECT * FROM Users WHERE name = ?      -- value bound separately, never executable
```

One-line method recall (for the morning)

1 ER: read sentence by sentence, weak entities, look-across cardinalities. 2 Relational: 1:N puts FK on N-side, N:M makes a table. 3 Canonical: singletons, left-reduce, remove redundant. 4 Keys: closures of optional-attribute combos, skip supersets. 5 BCNF: split on each FD whose LHS is not a superkey until every part is BCNF. 6 Anomalies: insert/delete/update from redundancy. 7 Conditions: parenthesize OR. 8 Joins: alias each role, join via bridge. 9 Aggregation: GROUP BY the entity, condition in HAVING. 10 Negation: NOT EXISTS the positive set. 11 For-all: double NOT EXISTS. 12 Serializability: precedence graph acyclic. 13 2PL: all locks before any unlock, hold to lock point. 14 Multigranularity: only IS/IX/S/X compatible pairs above. 15 Injection: parameterized queries separate code

from data.